



Uniwersytet Rzeszowski
Kolegium Nauk Przyrodniczych
Instytut Informatyki

Uniwersytet Rzeszowski
Kolegium Nauk Przyrodniczych
Instytut Informatyki

Praca projektowa programowanie obiektowe

System Zarządzania Warsztatem Samochodowym – WPF

Prowadzący:

mgr inż. Ewa Żesławska

Autor:

Sebastian Kuzyk

nr albumu:

131461

Kierunek: Informatyka, grupa lab 2

Rzeszów 2026

Spis treści

1	Streszczenie	3
2	Wprowadzenie i analiza problemu	4
2.1	Opis problemu	4
2.2	Cel projektu	4
2.3	Zakres funkcjonalności	4
2.4	Przegląd istniejących rozwiązań	4
3	Wymagania systemowe	5
3.1	Wymagania funkcjonalne	5
3.2	Wymagania нефункционалне	5
3.3	Założenia	6
4	Architektura i struktura aplikacji	6
4.1	Architektura ogólna	6
4.2	Struktura folderów	6
4.3	Wzorce projektowe	7
4.4	Walidacja danych	7
4.5	Obsługa błędów	7
5	Technologie wykorzystane w projekcie	8
5.1	Interfejs użytkownika	8
5.2	Dostęp do danych	8
5.3	Zestawienie pakietów NuGet	8
5.4	Uzasadnienie wyboru	8
6	Projekt bazy danych	8
6.1	Model danych	8
6.2	Diagram ERD	9
6.3	Tabele i relacje	9
6.4	Klucze główne i obce	9
6.5	Normalizacja	10
6.6	Konfiguracja kontekstu i danych startowych	10
6.7	Integralność	10
7	Implementacja aplikacji	11
7.1	Najważniejsze moduły	11
7.2	Logowanie i autoryzacja	11
7.3	Operacje CRUD	11
7.4	Reguła automatycznej zmiany statusu zlecenia	11
7.5	Nawigacja w panelach	12
7.6	Stylizacja interfejsu	12
7.7	Obsługa zegara	12

8	Prezentacja warstwy użytkowej projektu	13
8.1	Logowanie	13
8.2	Panel administratora	13
8.3	Panel mechanika	15
8.4	Panel recepcji	16
8.5	Panel magazyniera	17
8.6	Scenariusz pełnego przepływu	18
9	Harmonogram realizacji projektu	19
10	Instrukcja uruchomienia projektu	19
10.1	Wymagania środowiska	19
10.2	Konfiguracja środowiska	19
10.3	Uruchomienie z konsoli	19
10.4	Dane logowania testowego	20
11	Podsumowanie	20
11.1	Stopień realizacji założeń	20
11.2	Napotkane problemy	20
11.3	Kierunki dalszego rozwoju	20
	Bibliografia	20
	Spis rysunków	21
	Spis tabel	21
	Spis listingów	21

1 Streszczenie

Streszczenie (PL)

Przedmiotem projektu jest aplikacja desktopowa wspomagająca pracę warsztatu samochodowego. Celem było zaprojektowanie i zaimplementowanie systemu, który ułatwia rejestrację zleceń napraw, prowadzenie ewidencji klientów, pojazdów oraz magazynu części, a także rozdzielanie pracy między pracowników o różnych rolach. Aplikacja powstała w technologii WPF (C#, .NET 10) i komunikuje się z bazą SQLite przez Entity Framework Core. Interfejs zorganizowano wokół panelu bocznego z dashboardem dostosowanym do roli użytkownika. Rezultatem jest spójny, czytelny system z czterema panelami (administrator, mechanik, recepcja, magazynier), w którym dane przepływają w sposób zgodny z procesem biznesowym warsztatu — od rejestracji pojazdu, przez wykonanie napraw i zamówienie części, po wydanie pojazdu klientowi.

Abstract (EN)

The project delivers a desktop application that supports daily operations of a car repair shop. The goal was to design and implement a system that streamlines the registration of repair orders, the management of customers, vehicles and the parts inventory, and the distribution of tasks among employees with different roles. The application was built with WPF (C#, .NET 10) and communicates with a SQLite database through Entity Framework Core. The interface is organised around a sidebar with a role-specific dashboard. The result is a coherent, role-based system with four panels (administrator, mechanic, reception, warehouse keeper) that mirrors the business process of the workshop — from vehicle intake, through repair work and parts ordering, to the handover back to the customer.

2 Wprowadzenie i analiza problemu

2.1 Opis problemu

Małe i średnie warsztaty samochodowe prowadzą codzienną pracę często w sposób mieszany — część informacji zapisywana jest na papierze, część w arkuszach kalkulacyjnych, a kolejka napraw bywa komunikowana ustnie. Taka organizacja prowadzi do problemów: gubienia danych pojazdu, niepewności co do statusu naprawy, niespójnych stanów magazynowych oraz braku przejrzystości po stronie klienta. Praca, która powinna trwać dwa dni, przeciąga się, ponieważ mechanik nie wie, że części są wyczerpane, a magazynier nie wie, że są potrzebne.

2.2 Cel projektu

Celem projektu było zbudowanie aplikacji, która zastępuje rozproszone narzędzia jednym, spójnym systemem. Każdy pracownik ma własny widok dopasowany do swoich obowiązków, a dane przepływają między rolami w sposób naturalny — recepcja zakłada zlecenie, mechanik aktualizuje status zadań i zgłasza brakujące części, magazynier reaguje na zgłoszenia, administrator nadzoruje całość. System ma być prosty w obsłudze, a jednocześnie zapewniać integralność danych przez wspólną bazę SQLite.

2.3 Zakres funkcjonalności

Zakres prac obejmował:

- logowanie użytkowników z rozróżnieniem czterech ról,
- dashboard każdego panelu z kafelkami profilu i kluczowymi statystykami,
- rejestrację zlecenia naprawy razem z klientem, pojazdem i listą zadań naprawczych,
- zarządzanie zadaniami w zleceniu wraz z automatyczną zmianą statusu zlecenia,
- zarządzanie magazynem części (dodawanie, odbiór dostawy, wyszukiwanie),
- obsługę zgłoszeń o części (mechanik → magazynier) z dwoma wariantami: część z magazynu lub część do zamówienia,
- wydanie auta klientowi po zakończeniu naprawy,
- moduł administratora ze statystykami warsztatu i wykresami.

2.4 Przegląd istniejących rozwiązań

AutoFakir Polski system dla warsztatów oferujący moduły faktur, magazynu i kalendarza. Jest funkcjonalny, ale skierowany do większych firm, ma rozbudowany interfejs i wymaga subskrypcji. Dla małego warsztatu próg wejścia jest relatywnie wysoki.

Shop-Ware (US) Webowa platforma dla warsztatów z elektronicznymi kartami pracy i czatem z klientem. Oferuje rozwinięte funkcje, ale jest cały czas w chmurze i zorientowana na rynek amerykański — nazewnictwo procesów, integracje z dostawcami i waluta nie pasują do polskiego warsztatu.

Mitchell 1 ManagerSE Klasyczne rozwiązanie desktopowe dla warsztatów z bardzo bogatym zestawem funkcji (księgowość, integracja z VIN, raportowanie). System jest jednak złożony, drogi i wymaga dużego nakładu na wdrożenie.

Wnioski Każde z opisanych narzędzi pokrywa potrzeby dużego warsztatu, ale jest zbyt rozbudowane lub kosztowne dla małej firmy. Projekt *Warsztat Samochodowy WPF* celuje w niższą, w której najważniejsza jest prostota, przewidywalny przepływ pracy i lokalna baza danych pod kontrolą warsztatu.

3 Wymagania systemowe

3.1 Wymagania funkcjonalne

Tabela 1 przedstawia zbiór wymagań funkcjonalnych przypisanych do ról.

Tabela 1: Wymagania funkcjonalne aplikacji

ID	Opis
WF-01	Logowanie z rozróżnieniem ról (admin, mechanic, recepcja, magazynier).
WF-02	Edycja własnego profilu (login, hasło, telefon, e-mail).
WF-03	Recepcja: dodawanie klienta, pojazdu i nowego zlecenia z listą zadań.
WF-04	Recepcja: filtrowanie zleceń po statusie, podsumowanie kosztów, usuwanie zlecenia.
WF-05	Recepcja: oznaczanie pojazdu jako odebranego po zapłacie.
WF-06	Mechanik: lista własnych zleceń z filtrem statusu.
WF-07	Mechanik: szczegóły zadań ze znacznikami <i>Wykonane</i> / <i>Brakuje części</i> .
WF-08	Mechanik: automatyczna zmiana statusu zlecenia w zależności od stanu zadań.
WF-09	Mechanik: zgłoszenie zapotrzebowania na część (z magazynu lub do zamówienia).
WF-10	Magazynier: zarządzanie częściami (dodawanie, edycja stanu, dostawy).
WF-11	Magazynier: obsługa zgłoszeń od mechaników (zmiana statusu zgłoszenia).
WF-12	Administrator: zarządzanie pracownikami (CRUD).
WF-13	Administrator: dashboard z KPI i wykresami warsztatu.

3.2 Wymagania niefunkcjonalne

- **Platforma:** Windows 10/11, .NET 10 SDK.
- **Czas reakcji UI:** typowe operacje CRUD poniżej 200 ms na lokalnej bazie SQLite.
- **Integralność danych:** spójność klucza obcego między pojazdem, klientem, zleceniem i zadaniami.
- **Czytelność interfejsu:** jednolity sidebar, kolory dopasowane do roli, kontrast tekstu zgodny z dobrymi praktykami.

- **Lokalizacja:** pełna polska wersja językowa.
- **Brak zależności sieciowych:** aplikacja działa offline, baza jest plikiem SQLite.

3.3 Założenia

Aplikacja jest desktopowym narzędziem działającym w sieci wewnętrznej warsztatu, dlatego model bezpieczeństwa został świadomie ograniczony do prostego logowania na podstawie loginu i hasła. Hasła przechowywane są w postaci jawnej — jest to założenie wstępne, możliwe do zaostżenia w kolejnej iteracji. Plik bazy znajduje się obok aplikacji, a jego zabezpieczenie (kopia na zewnętrzny dysk, kontrola dostępu do katalogu) leży po stronie warsztatu.

4 Architektura i struktura aplikacji

4.1 Architektura ogólna

Aplikacja jest klasycznym desktopem trójwarstwowym:

- warstwa prezentacji — okna i dialogi WPF (XAML + code-behind),
- warstwa logiki domenowej — serwisy (LoginService, UserService) i bezpośrednie zapytania LINQ-to-Entities w okienkach,
- warstwa dostępu do danych — klasa CarWorkshopContext (Entity Framework Core) i modele encji w katalogu Models.

4.2 Struktura folderów

```

1 CarWorkshopWPF/
2 |-- App.xaml, App.xaml.cs           // konfiguracja WPF, style globalne
3 |-- CarWorkshopWPF.csproj           // definicja projektu, pakiety NuGet
4 |-- car_workshop.db                 // baza SQLite z danymi startowymi
5 |-- Data/
6 |   '-- CarWorkshopContext.cs       // DbContext, mapowanie encji
7 |-- Models/                         // encje EF Core
8 |   |-- User.cs, Customer.cs, Vehicle.cs
9 |   |-- RepairOrder.cs, RepairTask.cs
10 |   |-- Part.cs, OrderPart.cs, PartRequest.cs
11 |   '-- ServiceType.cs
12 |-- Services/
13 |   |-- LoginService.cs             // autentykacja
14 |   '-- UserService.cs              // CRUD użytkowników
15 '-- Views/
16 |   |-- LoginWindow                 // logowanie
17 |   |-- AdminWindow                 // panel administratora
18 |   |-- MechanicWindow              // panel mechanika
19 |   |-- RecepcjaWindow              // panel recepcji
20 |   |-- MagazynierWindow            // panel magazyniera
21 '-- (dialogi pomocnicze: zadania, czesci, status)

```

Listing 1: Struktura katalogów projektu

4.3 Wzorce projektowe

W projekcie wykorzystano następujące wzorce:

- **Repository / Unit of Work** — zapewnia je sam EF Core (DbContext agreguje DbSet i operuje w jednej transakcji SaveChanges).
- **Krótkożyjące konteksty (scoped DbContext)** — każdy widok tworzy własny kontekst (using var ctx = new CarWorkshopContext()), co minimalizuje konflikty śledzenia encji i upraszcza zarządzanie zasobami.
- **Code-Behind (Page Controller)** — każdy XAML ma dedykowaną klasę logiki, która obsługuje zdarzenia interfejsu.
- **Strategy w nawigacji** — po zalogowaniu wybór panelu odbywa się przez wyrażenie switch na roli użytkownika (Listing 2).

```
1 private void OpenPanelForRole (Models.User user)
2 {
3     Window? panel = user.Role switch
4     {
5         "admin"      => new AdminWindow(user),
6         "mechanic"   => new MechanicWindow(user),
7         "recepccja"  => new RecepccjaWindow(user),
8         "magazynier" => new MagazynierWindow(user),
9         _            => null
10    };
11    if (panel != null) { panel.Show(); this.Close(); }
12 }
```

Listing 2: Routing roli na panel po logowaniu

4.4 Walidacja danych

Walidacja prowadzona jest w warstwie prezentacji (przed wysłaniem do serwisu lub kontekstu). Sprawdzane są m.in.:

- obecność wymaganych pól (login, imię, nazwisko, marka, model, rejestracja),
- poprawność e-maila (znak @) i numeru telefonu (9 cyfr),
- dodatnia liczba dla cen i ilości,
- unikalność loginu przy dodawaniu pracownika,
- sensowny rok produkcji (1900 – bieżący rok + 1) w przypadku pojazdu.

4.5 Obsługa błędów

Operacje na bazie danych są opakowane w blok try-catch. Komunikat dla użytkownika wyświetlany jest przez MessageBox z czytelną informacją — bez technicznego stack trace'u. W przypadku naruszenia klucza obcego (np. próba usunięcia pracownika z aktywnymi zleceniami) komunikat informuje, że obiekt posiada powiązania.

5 Technologie wykorzystane w projekcie

5.1 Interfejs użytkownika

WPF Wybór WPF wynikał z dwóch przesłanek. Po pierwsze, technologia jest natywna dla Windows i nie wymaga dodatkowych runtime'ów poza .NET. Po drugie, separacja deklaratywnego XAML od kodu w C# pozwoliła oddzielić wygląd okien od logiki — to przyspieszyło iteracje nad designem sidebara i kolorystyką poszczególnych paneli.

LiveChartsCore Biblioteka `LiveChartsCore.SkiaSharpView.WPF` w wersji `2.0.0-rc5.4` zapewnia wykresy w panelu administratora (kolumnowy i kołowy). Wybrana ze względu na natywną obsługę WPF, dobre osadzenie w XAML (`lvc:CartesianChart`, `lvc:PieChart`) i wsparcie dla nowoczesnych wersji .NET.

5.2 Dostęp do danych

Entity Framework Core 10 + SQLite Plik `car_workshop.db` (SQLite) jest jedynym nośnikiem danych i znajduje się w katalogu projektu. Połączenie konfigurowane jest w `CarWorkshopContext` przez prostą ścieżkę względną. Modele EF korzystają z atrybutów `[Table]` oraz `[Column]`, dzięki czemu w bazie obowiązuje konwencja `snake_case` (zgodna ze standardem SQL), niezależnie od konwencji `PascalCase` używanej w kodzie C#.

5.3 Zestawienie pakietów NuGet

Tabela 2: Wykorzystane pakiety NuGet

Pakiet	Wersja	Zastosowanie
Microsoft.EntityFrameworkCore.Sqlite	10.0.8	Dostęp do SQLite
Microsoft.EntityFrameworkCore.Tools	10.0.8	Migracje (dev-time)
LiveChartsCore.SkiaSharpView.WPF	2.0.0-rc5.4	Wykresy w panelu admin

5.4 Uzasadnienie wyboru

Stos technologiczny celowo trzyma się minimum: WPF + EF Core + jedna biblioteka do wykresów. Dzięki temu projekt jest łatwy do uruchomienia, nie wymaga konfiguracji zewnętrznych usług (serwer bazy, brokery, chmura) i działa od razu po pobraniu repozytorium — plik bazy danych jest dołączony do projektu i kopiowany do katalogu wynikowego przy każdym buildzie.

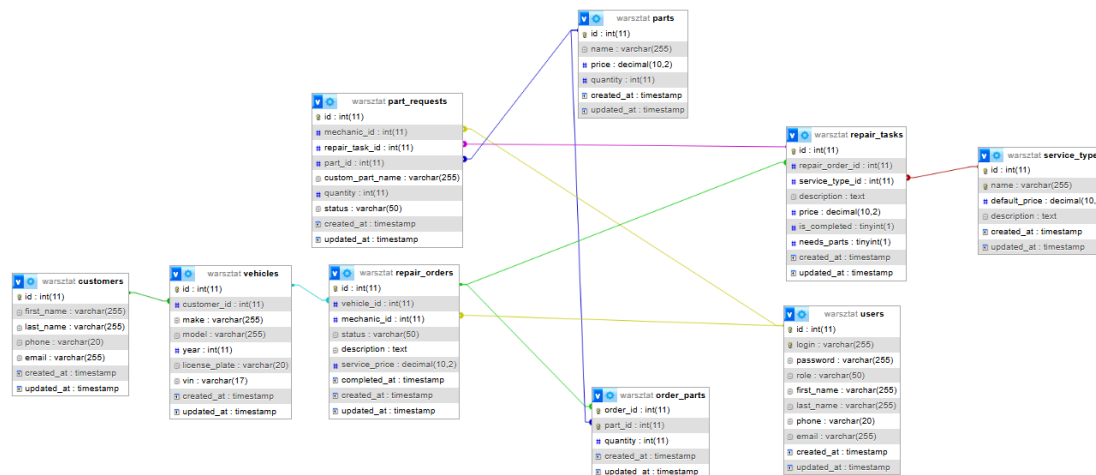
6 Projekt bazy danych

6.1 Model danych

Schemat bazy odzwierciedla naturalny obieg dokumentów w warsztacie. Klient jest właścicielem pojazdu, na pojeździe zakładane jest zlecenie naprawy, w ramach zlecenia powstają zadania powiązane z typami usług, a zadania wymagają części z magazynu.

Tabela `order_parts` pełni rolę łącznika dla zużycia części w zleceniu, a `part_requests` przechowuje komunikację mechanika z magazynierem.

6.2 Diagram ERD



Rysunek 1: Schemat ERD

6.3 Tabele i relacije

Tabela 3: Główne tabele bazy danych

Tabela	Przeznaczenie i kluczowe pola
users	pracownicy: login, password, role, dane kontaktowe
customers	klienci: imię, nazwisko, telefon, e-mail, data utworzenia
vehicles	pojazdy: customer_id, marka, model, rok, rejestracja, VIN
repair_orders	zlecenia: vehicle_id, mechanic_id, status, daty
repair_tasks	zadania w zleceniu: opis, cena, is_completed, needs_parts
service_types	cennik usług (np. wymiana oleju, klocków)
parts	części magazynowe: nazwa, cena, ilość
order_parts	klucz złożony (order_id, part_id) — użycie części
part_requests	zgłoszenia mechanika: part_id albo custom_part_name, status

6.4 Klucze główne i obce

- Każda tabela — poza `order_parts` — ma sztuczny klucz id typu `INTEGER PRIMARY KEY`.
- `order_parts` ma klucz złożony (`order_id`, `part_id`) skonfigurowany w `OnModelCreating`.

- Klucze obce: `vehicles.customer_id` → `customers`, `repair_orders.vehicle_id` → `vehicles`, `repair_orders.mechanic_id` → `users`, `repair_tasks.repair_order_id` → `repair_orders`, `repair_tasks.service_type_id` → `service_types`, `order_parts.order_id` → `repair_orders`, `order_parts.part_id` → `parts`, `part_requests.mechanic_id` → `users`, `part_requests.repair_task_id` → `repair_tasks`, `part_requests.part_id` → `parts`.

6.5 Normalizacja

Schemat jest w trzeciej postaci normalnej. Dane redundantne nie są przechowywane: koszt zlecenia wyliczany jest na podstawie sumy cen zadań i części, status zlecenia oblicza się z bieżącego stanu zadań, a cena domyślna usługi pochodzi z tabeli `service_types` (a nie jest kopiowana do każdego zadania).

6.6 Konfiguracja kontekstu i danych startowych

Listing 3 pokazuje konfigurację `DbContext` i sposób wskazania pliku bazy. Plik `car_workshop.db` jest częścią projektu i przy każdym buildzie kopiowany do katalogu wyjściowego (reguła `<CopyToOutputDirectory>` w `CarWorkshopWPF.csproj`). Dzięki temu po pobraniu repozytorium aplikacja uruchamia się od razu, z gotowym zestawem danych przykładowych (kont użytkowników, cennika usług oraz początkowego stanu magazynu).

```
1 public class CarWorkshopContext : DbContext
2 {
3     public DbSet<ServiceType> ServiceTypes { get; set; }
4     public DbSet<User> Users { get; set; }
5     public DbSet<Customer> Customers { get; set; }
6     public DbSet<Vehicle> Vehicles { get; set; }
7     public DbSet<RepairOrder> RepairOrders { get; set; }
8     public DbSet<RepairTask> RepairTasks { get; set; }
9     public DbSet<Part> Parts { get; set; }
10    public DbSet<OrderPart> OrderParts { get; set; }
11    public DbSet<PartRequest> PartRequests { get; set; }
12
13    protected override void OnConfiguring(DbContextOptionsBuilder
14        options)
15        => options.UseSqlite("Data Source=car_workshop.db");
16
17    protected override void OnModelCreating(ModelBuilder modelBuilder)
18    {
19        modelBuilder.Entity<OrderPart>()
20            .HasKey(op => new { op.OrderId, op.PartId });
21    }
22 }
```

Listing 3: Konfiguracja `DbContext` — lokalna baza SQLite

6.7 Integralność

Integralność danych zapewniają: ograniczenia klucza obcego SQLite, kontrola statusu po stronie aplikacji (statusy są walidowane z listy) oraz transakcyjność `SaveChanges` — każda operacja zapisuje wszystkie zmiany lub żadne.

7 Implementacja aplikacji

7.1 Najważniejsze moduły

Aplikacja składa się z czterech głównych modułów funkcjonalnych odpowiadających rolom oraz z modułu wspólnego (logowanie, edycja profilu, dialogi pomocnicze). Każdy moduł działa w sidebar-owym layoucie z górnym paskiem (data, godzina) i przełączanymi ekranami w `TabControl` ukrytych nagłówków.

7.2 Logowanie i autoryzacja

Logowanie sprawdza login i hasło w tabeli `users`. Po sukcesie otwierany jest panel właściwy dla roli, a okno logowania jest zamykane. Listing 4 pokazuje rdzeń uwierzytelniania.

```
1 public static User? AuthenticateUser(string login, string password)
2 {
3     using var context = new CarWorkshopContext();
4     return context.Users.FirstOrDefault(
5         u => u.Login == login && u.Password == password);
6 }
```

Listing 4: Uwierzytelnianie użytkownika

W obecnej wersji projektu hasła przechowywane są jako tekst jawny — jest to świadome uproszczenie wynikające z założenia, że aplikacja działa w sieci wewnętrznej warsztatu, a fizyczny dostęp do pliku bazy jest kontrolowany. Wprowadzenie hashowania (PBKDF2 lub BCrypt) zostało wskazane jako pierwszy element w kierunkach dalszego rozwoju (sekcja *Podsumowanie*).

7.3 Operacje CRUD

Operacje CRUD realizowane są bezpośrednio przez `DbContext`. Listing 5 pokazuje dodawanie pracownika z walidacją oraz kontrolą unikalności loginu.

```
1 using var ctx = new CarWorkshopContext();
2 if (ctx.Users.Any(u => u.Login == login)) {
3     MessageBox.Show("Taki login już istnieje!");
4     return;
5 }
6 ctx.Users.Add(new User {
7     Login = login, Password = pwd, Role = role,
8     FirstName = firstName, LastName = lastName,
9     Phone = phone, Email = email
10 });
11 ctx.SaveChanges();
```

Listing 5: Dodawanie pracownika z walidacją

7.4 Reguła automatycznej zmiany statusu zlecenia

Najciekawszym fragmentem logiki jest automatyczna zmiana statusu zlecenia naprawy w zależności od stanu jego zadań. Reguła znajduje się w dialogu szczegółów zadań mechanika (Listing 6).

```
1 bool anyNeedsParts = order.RepairTasks.Any(t => t.NeedsParts);
2 int completedCount = order.RepairTasks.Count(t => t.IsCompleted);
3 int total          = order.RepairTasks.Count;
4
5 string newStatus;
6 if (anyNeedsParts)          newStatus = "Czeka na części";
7 else if (completedCount == total) {
8     newStatus = "Gotowe do odbioru";
9     order.CompletedAt = DateTime.Now;
10 }
11 else if (completedCount > 0) newStatus = "W trakcie";
12 else                        newStatus =
13     order.Status == "W trakcie" ? "W trakcie" : "Nierozpoczęte";
14
15 order.Status = newStatus;
```

Listing 6: Auto-status zlecenia w TaskDetailsDialog

7.5 Nawigacja w panelach

Każde okno paneli ma sidebar w Border po lewej stronie i TabControl z ukrytymi nagłówkami po prawej. Przełączanie zakładek odbywa się przez `stackedWidget.SelectedIndex`, a aktywny przycisk podświetlany jest stylem `NavButtonActive` (Listing 7).

```
1 private void SetActiveButton(Button active)
2 {
3     foreach (var btn in new[] { buttonMain, buttonUsers,
4                                 buttonClients, buttonRepairs })
5     {
6         btn.Style = btn == active
7             ? (Style)FindResource("NavButtonActive")
8             : (Style)FindResource("NavButton");
9     }
10 }
```

Listing 7: Aktywny przycisk sidebara

7.6 Stylizacja interfejsu

Globalne style znajdują się w `App.xaml`: kolory przycisków akcji (sukces, ostrzeżenie, niebezpieczeństwo, neutralny), karta KPI dla dashboardu, jednolity `DataGrid` z naprzemiennym podświetleniem wierszy. Każdy panel ma swój kolor sidebara, dzięki czemu pracownik na pierwszy rzut oka widzi, w jakim trybie pracuje (granat dla mechanika, fiolet dla recepcji, brąz dla magazyniera, ciemny granat dla administratora).

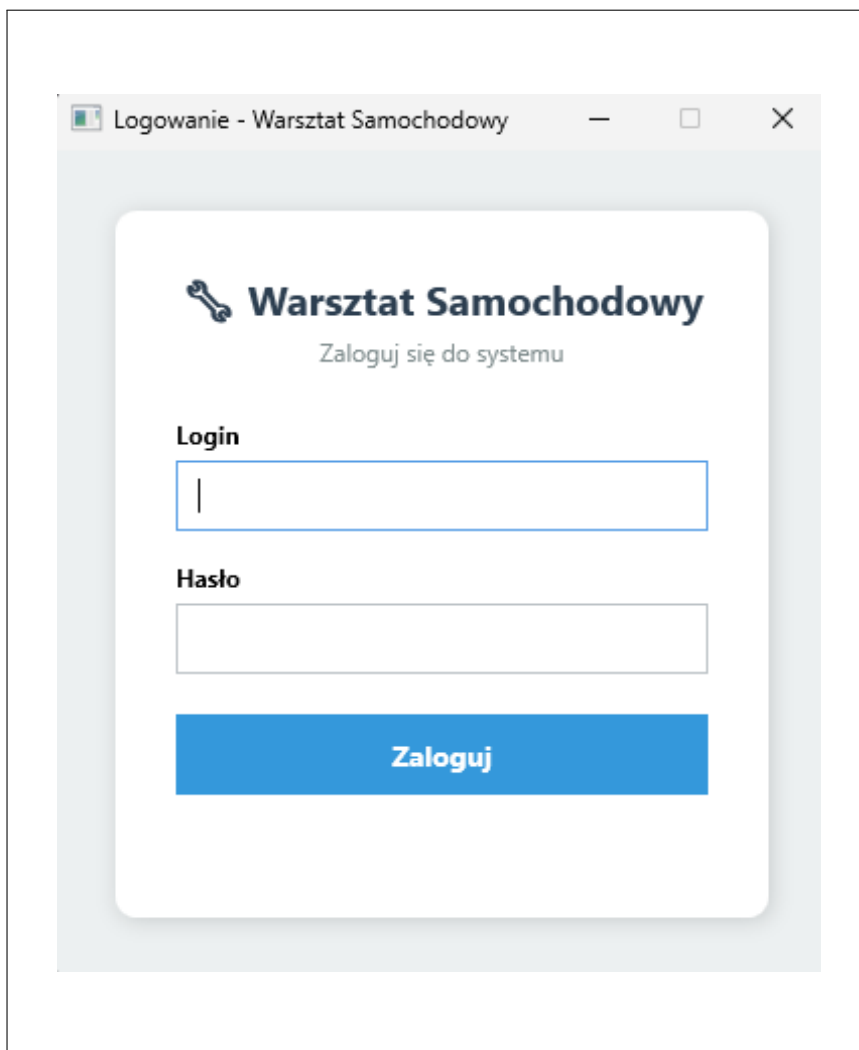
7.7 Obsługa zegara

W każdym panelu działa `DispatcherTimer` aktualizujący datę i godzinę co sekundę. Timer jest zatrzymywany w zdarzeniu `Closed`, co zapobiega wyciekowi zasobów po zamknięciu okna.

8 Prezentacja warstwy użytkowej projektu

8.1 Logowanie

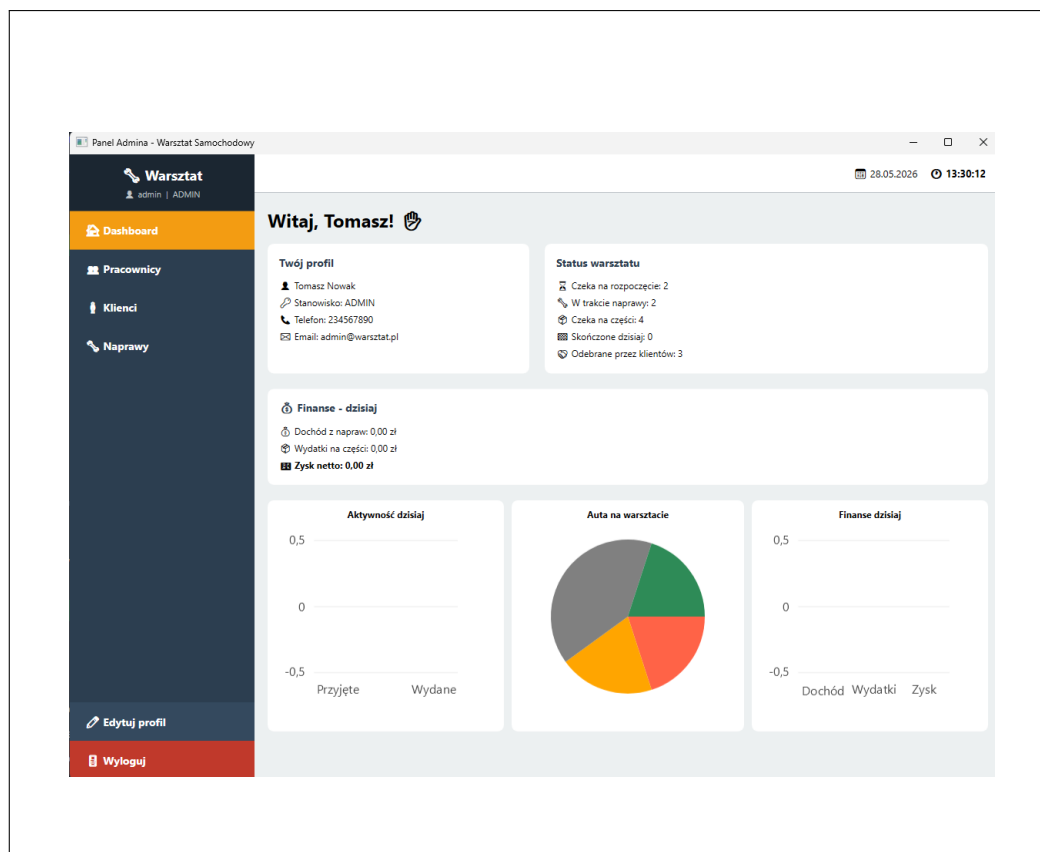
Pierwszym ekranem aplikacji jest formularz logowania (rys. 2). Po wpisaniu poprawnych danych użytkownik przekierowywany jest do panelu właściwego dla swojej roli, a w razie błędu pojawia się czerwony komunikat.



Rysunek 2: Ekran logowania

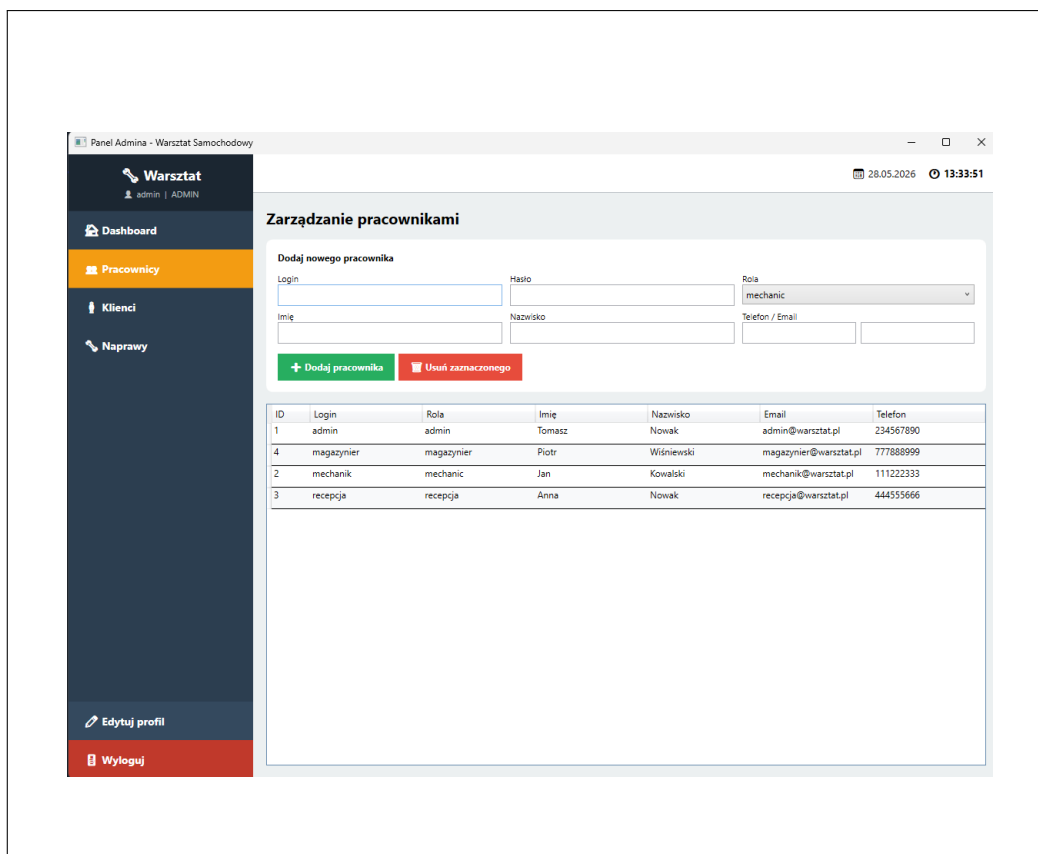
8.2 Panel administratora

Administrator widzi dashboard z dwoma kartami (profil + status warsztatu) oraz blok finansowy. Niżej znajdują się trzy wykresy LiveCharts: aktywność dzisiaj (kolumnowy), auta na warsztacie (kołowy), finanse dzisiaj (kolumnowy).



Rysunek 3: Dashboard administratora

W zakładce *Pracownicy* administrator zarządza listą użytkowników (rys. 4). Formularz dodawania znajduje się nad tabelą, a usuwanie chronione jest przed usunięciem samego siebie.



Rysunek 4: Zarządzanie pracownikami

8.3 Panel mechanika

Mechanik ma dostęp do swoich zleceń, magazynu (tylko podgląd) oraz listy własnych zgłoszeń o części. Po wybraniu zlecenia może otworzyć dialog szczegółów zadań (rys. 5), w którym zaznacza *Wykonane* lub *Brakuje części*. Zaznaczenie tego drugiego otwiera formularz wyboru części z magazynu lub wpisania nazwy do zamówienia.

Szczegóły zlecenia

TEST TEST (TEST123)

Klient: asd asd
Status: Czekaj na części
Utworzono: 15.05.2026 14:28

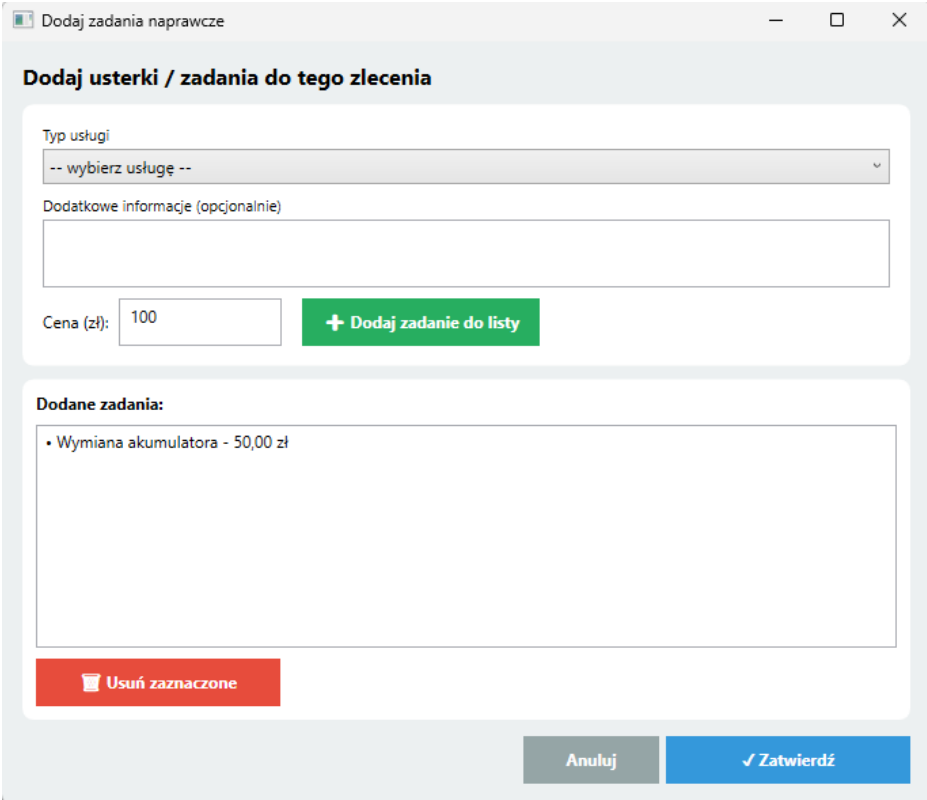
Zadanie 1	800,00 zł
Wymiana rozrządu	
☒ ✓ Wykonane	☐ 🛠 Brakuje części
Zadanie 2	250,00 zł
Naprawa klimatyzacji	
☒ ✓ Wykonane	☐ 🛠 Brakuje części
Zadanie 3	250,00 zł
Wymiana amortyzatorów	
☐ ✓ Wykonane	☒ 🛠 Brakuje części

Zapisz zmiany + Dodaj część do zlecenia Zamknij

Rysunek 5: Szczegóły zadań w zleceniu

8.4 Panel recepcji

Recepcja jest sercem operacyjnym aplikacji — to tutaj powstają nowe zlecenia. Formularz nowego zlecenia (rys. 6) zbiera dane klienta, pojazdu i mechanika, a po naciśnięciu *Dalej* pojawia się dialog dodawania zadań, w którym dla każdej usługi z cennika można podać dodatkowe informacje i cenę.

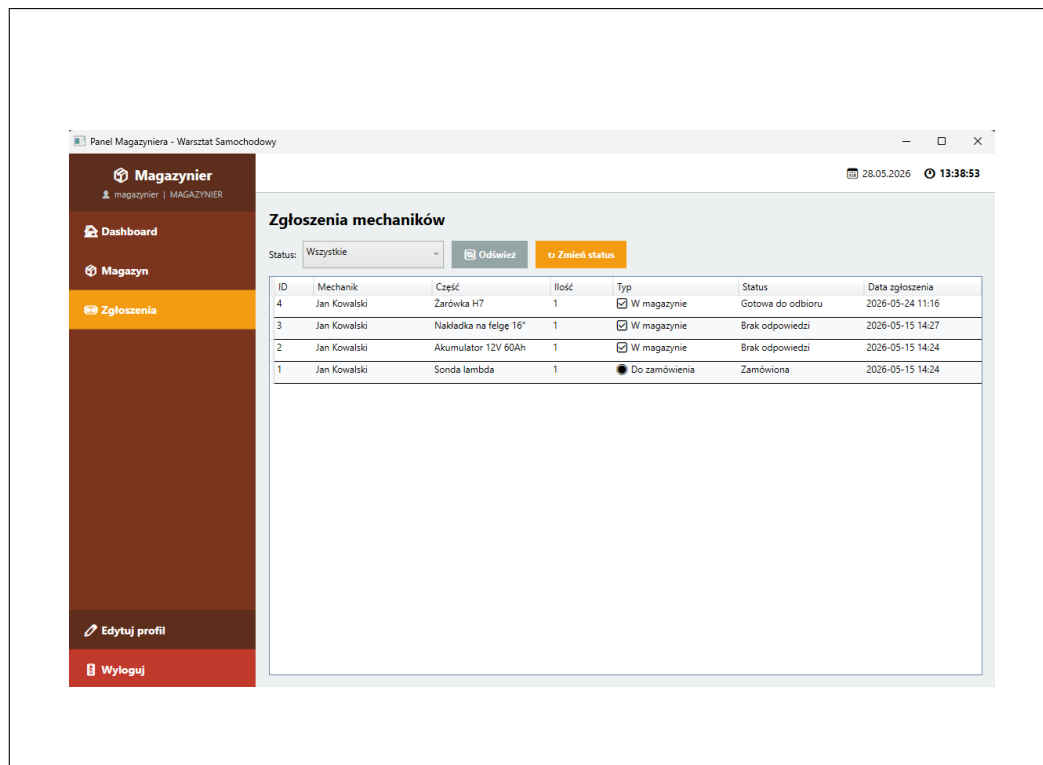


Rysunek 6: Tworzenie nowego zlecenia

W zakładce *Auta do oddania* recepcja widzi tylko zlecenia o statusie *Gotowe do odbioru* z wyliczoną sumą do zapłaty i może kliknąć *Oznacz jako odebrane* po zainkasowaniu płatności.

8.5 Panel magazyniera

Magazynier zarządza częściami i odpowiada na zgłoszenia (rys. 7). Lista zgłoszeń odróżnia kolorystycznie zgłoszenia do przygotowania (część jest w bazie) od tych do zamówienia (część niestandardowa).



Rysunek 7: Obsługa zgłoszeń o części

8.6 Scenariusz pełnego przepływu

1. Recepcjonistka tworzy zlecenie dla nowego klienta i pojazdu, dodając trzy zadania (np. wymiana oleju, klocków, przegląd).
2. Mechanik widzi zlecenie w swoim panelu ze statusem *Nierozpoczęte*.
3. Mechanik otwiera szczegóły, oznacza pierwsze zadanie jako wykonane (status zmienia się na *W trakcie*).
4. Drugie zadanie wymaga klocków — mechanik zaznacza *Brakuje części*, wybiera z magazynu i wysyła zgłoszenie. Status zmienia się na *Czeka na części*.
5. Magazynier zatwierdza zgłoszenie, zmienia jego status na *Gotowa do odbioru*.
6. Mechanik dodaje część do zlecenia, kończy pozostałe zadania. Status automatycznie przechodzi na *Gotowe do odbioru*.
7. Recepcja widzi pojazd w zakładce *Auta do oddania*, generuje podsumowanie kosztów (zadania + części) i po zapłacie oznacza pojazd jako *Odebrane*.
8. Administrator w dashboardzie widzi nowy słupek na wykresie finansów oraz spadek liczby aut na warsztacie.

9 Harmonogram realizacji projektu

Tabela 4: Harmonogram realizacji projektu

Etap	Zakres prac	Czas
1	Analiza wymagań, wstępny model danych, projekt UI	2 tydz.
2	Projekt schematu bazy SQLite, modele EF Core, kontekst	3 tydz.
3	Logowanie, sidebar, dashboard administratora	1 tydz.
4	Panele recepcji i mechanika z pełnym przepływem zlecenia	2 tyg.
5	Panel magazyniera, system zgłoszeń o części	2 tydz.
6	Wykresy LiveCharts, polerowanie UI, ujednolicenie stylów	1 tydz.
7	Testy ręczne, dane przykładowe, naprawa błędów	1 tydz.
8	Dokumentacja	2 tydz.

10 Instrukcja uruchomienia projektu

10.1 Wymagania środowiska

- Windows 10 lub 11 (x64),
- .NET 10 SDK (Windows Desktop),
- przestrzeń dyskowa: ~200 MB (z pakietami NuGet).

10.2 Konfiguracja środowiska

1. Pobranie .NET 10 SDK z <https://dotnet.microsoft.com/download>.
2. Sklonowanie repozytorium projektu na dysk lokalny.
3. Uruchomienie aplikacji ze skryptu lub z konsoli (poniżej).

Plik bazy `car_workshop.db` z danymi przykładowymi jest dołączony do repozytorium i kopiowany do katalogu wynikowego przy buildzie — nie wymaga ręcznej konfiguracji.

10.3 Uruchomienie z konsoli

```
1 cd CarWorkshopWPF
2 dotnet restore
3 dotnet build
4 dotnet run
```

Listing 8: Komendy uruchamiania

Alternatywnie można dwukrotnie kliknąć `uruchom.bat` (CMD) lub `uruchom.ps1` (PowerShell), które automatyzują powyższe kroki.

10.4 Dane logowania testowego

Tabela 5: Konta testowe

Rola	Login	Hasło
Administrator	admin	admin123
Mechanik	mechanik	mechanik123
Recepcja	recepcja	recepcja123
Magazynier	magazynier	magazynier123

11 Podsumowanie

11.1 Stopień realizacji założeń

Zrealizowane zostały wszystkie wymagania funkcjonalne z tabeli 1: cztery panele ról, rejestracja klientów i zleceń, system zgłoszeń części z dwoma wariantami (część z bazy / niestandardowa), automatyczna zmiana statusu zlecenia, dashboard z wykresami w panelu administratora, podsumowanie kosztów dla recepcji. Aplikacja jest w pełni samodzielna — po sklonowaniu repozytorium uruchamia się bez dodatkowej konfiguracji, korzystając z dołączonego pliku bazy SQLite z danymi przykładowymi.

11.2 Napotkane problemy

- **Mapowanie konwencji nazw** — baza zaprojektowana w konwencji `snake_case` (zgodnej ze standardem SQL), a kod C# w `PascalCase`. Rozwiązanie: atrybuty `[Table]` i `[Column]` w każdej encji definiujące jawne mapowanie.
- **Klucz złożony `order_parts`** — EF Core nie wykrywa go automatycznie. Rozwiązanie: jawna konfiguracja w `OnModelCreating` przez `HasKey(...)`.
- **Wersja LiveCharts** — początkowo wybrana wersja 2.0.4 nie była dostępna pod docelowy framework; poprawnie działa wersja 2.0.0-rc5.4.
- **Polskie znaki w XAML** — znak `<` w atrybucie `Content` traktowany jest jak początek znacznika. Rozwiązanie: encja `<`.
- **Synchronizacja z procesem aplikacji** — przy częstej przebudowie projektu plik `.exe` bywa zablokowany. Rozwiązanie: zamykanie aplikacji przed kolejnym buildem.

11.3 Kierunki dalszego rozwoju

- **Hashowanie haseł** (PBKDF2 lub BCrypt) z migracją istniejących kont.
- **Eksport raportu zlecenia do PDF** jako gotowy dokument fakturowy.
- **Pełne MVVM** — wprowadzenie ViewModeli i bindingów tam, gdzie obecnie dominuje code-behind.
- **Powiadomienia o niskich stanach** jako toast w panelu magazyniera.
- **API REST** dla mobilnego klienta dla doradców serwisowych.

Bibliografia

- [1] Microsoft Learn, *Windows Presentation Foundation (WPF) Documentation*. <https://learn.microsoft.com/dotnet/desktop/wpf/>
- [2] Microsoft Learn, *Entity Framework Core — Overview*. <https://learn.microsoft.com/ef/core/>
- [3] SQLite Consortium, *SQLite Documentation*. <https://www.sqlite.org/docs.html>
- [4] beto-rodriguez, *LiveCharts2 — documentation and samples*. <https://livecharts.dev/>
- [5] Microsoft Learn, *XAML overview (WPF .NET)*. <https://learn.microsoft.com/dotnet/desktop/wpf/xaml/>
- [6] Microsoft Learn, *C# language reference*. <https://learn.microsoft.com/dotnet/csharp/>

Spis rysunków

1	Schemat ERD	9
2	Ekran logowania	13
3	Dashboard administratora	14
4	Zarządzanie pracownikami	15
5	Szczegóły zadań w zleceniu	16
6	Tworzenie nowego zlecenia	17
7	Obsługa zgłoszeń o części	18

Spis tabel

1	Wymagania funkcjonalne aplikacji	5
2	Wykorzystane pakiety NuGet	8
3	Główne tabele bazy danych	9
4	Harmonogram realizacji projektu	19
5	Konta testowe	20

Spis listingów

1	Struktura katalogów projektu	6
2	Routing roli na panel po logowaniu	7
3	Konfiguracja DbContext — lokalna baza SQLite	10
4	Uwierzytelnianie użytkownika	11
5	Dodawanie pracownika z walidacją	11
6	Auto-status zlecenia w <code>TaskDetailsDialog</code>	12
7	Aktywny przycisk sidebara	12
8	Komendy uruchamiania	19